

Performance Evaluation of Tensorflow Parallel Strategies for Deep Learning Training on HPC Clusters

Rayan Raddatz de Matos¹, Kenichi Brumato¹, Marcelo Cardoso Oliveira Gulart¹

¹ Institute of Informatics, Federal University of Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

{rayan.raddatz, kbrumati, mcogulart}@inf.ufrgs.br

Abstract. *The growing complexity of Deep Learning models makes training on single devices difficult, requiring distributed strategies like Data Parallelism. However, this approach creates critical synchronization challenges between processing units. This work evaluates TensorFlow parallel strategies on an HPC cluster to analyze how application factors contribute to time reduction. The study compares the standard synchronous method against a proposed Custom Training Loop designed to reduce bottleneck by synchronizing gradients less frequently. Experiments using ResNet50 and EfficientNetB0 models show that the Custom Training Loop significantly lowers communication overhead compared to the standard approach. Additionally, results indicate that increasing the batch size consistently improves performance. The authors conclude that correct configuration allows for performance gains in distributed training, even on networks with lower throughput.*

1. Introduction

In the last decade, Deep Learning has established itself as the state-of-the-art for solving complex problems, such as computer vision and natural language processing. Given the progress in algorithmic precision, there has been an exponential growth in the complexity of neural network architectures and the volume of datasets. This scenario imposes a massive computational load, making the training of robust models on a single processing unit a difficult task in terms of time and resources.

To mitigate these challenges, distributed training has become a standard practice, with emphasis on the Data Parallelism strategy. In this approach, a complete replica of the model is instantiated on each processing unit, and the global dataset is partitioned into smaller, locally-focused batches. Although effective, this technique introduces critical synchronization challenges, as locally calculated gradients must be aggregated and communicated among all devices at every training step, typically through All-Reduce operations.

Given this fact, in this work we decided to analyze data parallelism for AI training using TensorFlow in a HPC cluster to investigate how the factors related to the application contribute to time reduction in a parallel scenario.

Software and Data Availability. We endeavor to make our analysis reproducible for a better science. We made available a companion material hosted in a public GitHub repository at <https://github.com/kenichi220/comp-sys-perf-analysis-final-work/tree/main>. Our companion contains the source code of this article and the software necessary to handle the created datasets. We also include instructions to run the experiment and figures.

2. Related Work

[This should be a small section] [Marcelo will write about related works]

3. Methods and Materials

3.1. Hardware and Software Configuration

The experiments were run using Python 3.11.2 and TensorFlow 2.15 as newer versions showed errors when trying to train using more than one node with the `.fit` and `MultiWorkerMirroredStrategy`. We used `pip` to manage the python libraries and ensure reproducibility. The NVIDIA drivers version was 550.54.15 and CUDA 12.4. We used up to three computational nodes connected with a network of 1G, each node contains an Intel(R) Core(TM) i7-14700KF processor at 3.40 GHz with 28 threads and 20 cores, 96 GB DDR5 RAM and NVIDIA GeForce RTX 4070 with 8GB memory accelerator.

3.2. Custom Training Loop

The expected behavior of training with the `MultiWorkerMirroredStrategy` would be to have a lower training time as the the number of nodes is increased. However, when performing distributed training in low bandwidth networks one common problem that surfaces is the synchronization overhead, resulting in a higher training time as we increase the number of nodes. This happens because the gradients and weights of the model are synchronized between all nodes when a batch is computed when using the `model.fit()` method. To mitigate this problem, we proposed a Custom Training Loop that instead of synchronizing the parameters between nodes in every batch, only synchronizing the gradients at a interval of batches, this then results to lower synchronization overhead. To this work we empirically defined this interval of batches as 4 batches.

3.3. Design of Experiments (DoE)

We selected two different Deep Learning models to train, one big and one small: **ResNet50** [He et al. 2015] as our big model, with ~25.6M parameters, and **Efficient-NetB0** [Tan and Le 2019] as our small model, with ~5.3M parameters. The two networks were trained using the TinyImageNet-200, a subset of the ImageNet [Deng et al. 2009] containing 100000 downsized to a 64x64 colored image divided among 200 classes. We created a full factorial for each model, varying batch size (with levels 32, 64 and 100), the number of nodes used (1, 2 or 3) and the type of the training ("ctl" for Custom Training Loop and "sync" for training with the `model.fit` method). This is a total of 24 (2x3x3x2) different executions, each execution as replicated 5 times in a random order. As output variable we retrieved the training time.

4. Results

Figure 1 show a full view of our results. The graph presents the mean training time in minutes as a function of the number of nodes, faceted by type and batch size and with the color representing the model. Each bar represents the mean for the five executions for each configuration. The black transparent dots represents our observations. This allow us to see two main results: The first one is that the inclusion of more then one node introduces the necessity of communication, also introducing communication overhead. This overhead can be better seen with the smaller batches, as the computation is faster and

more communication is needed. We can also see that the ResNet50 shows a higher time then the EfficientNetB0 in this cases. This happens because all the parameters needed to be synchronized after every batch, and the ResNet50 is almost 5 times bigger than the EfficientNetB0. The second one is that the ctl showed a considerable smaller communication overhead even for small batch sizes. The ctl showed **OOM** errors when running with the batch size 100 for training the EfficientNetB0 model.

After the introduction of overhead by the inclusion of a second node, adding more nodes seemed to decrease the training time. We suppose that even with a low bandwidth network, with enough nodes the time gained by the distributed computing can overcome the overhead and show better results then running it in a single machine.

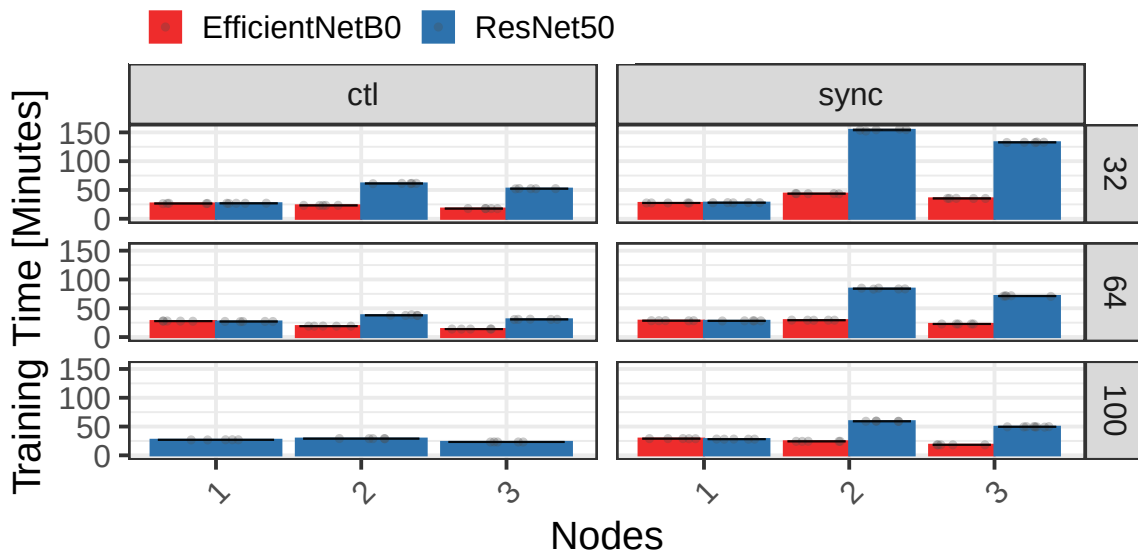


Figure 1. Mean of experimental results.

4.1. The impact of the number of nodes and batch size

Figure 2 shows the mean training time grouped by node count. This enable us to see that in fact the increase of more then two machines results in a lower training time even though this gain is small. Figure 3 shows also an interesting result, we can see that increase of the batch size consistently show performance gains, while also decreasing the variance. This means that conforms the batch size grows the training time is almost the same despite the node count.

5. Conclusion

We could conclude that the network and the model size have a huge importance in the distributed training, but that even with low bandwidth network it is possible to gain performance when training by correctly configuring the environment and model parameters. As future work we intend to deep investigate the impact of the network and the model size in the distributed training.

Acknowledgements

We would like to express our gratitude to Professor Dr. Lucas Mello Schnorr for his guidance and the valuable insights shared regarding performance analysis. We also would

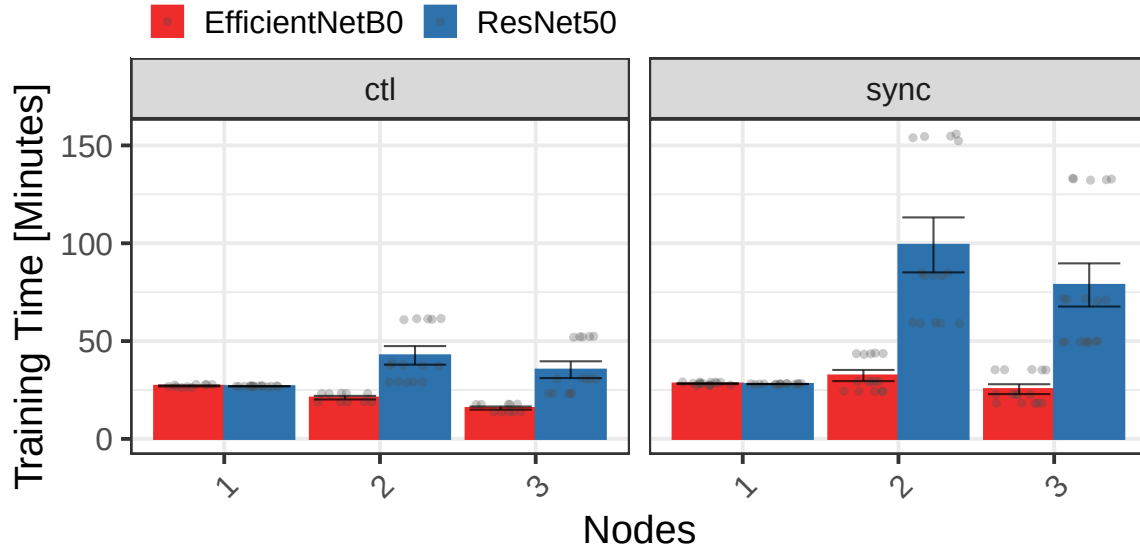


Figure 2. Mean of experimental results grouped by node count.

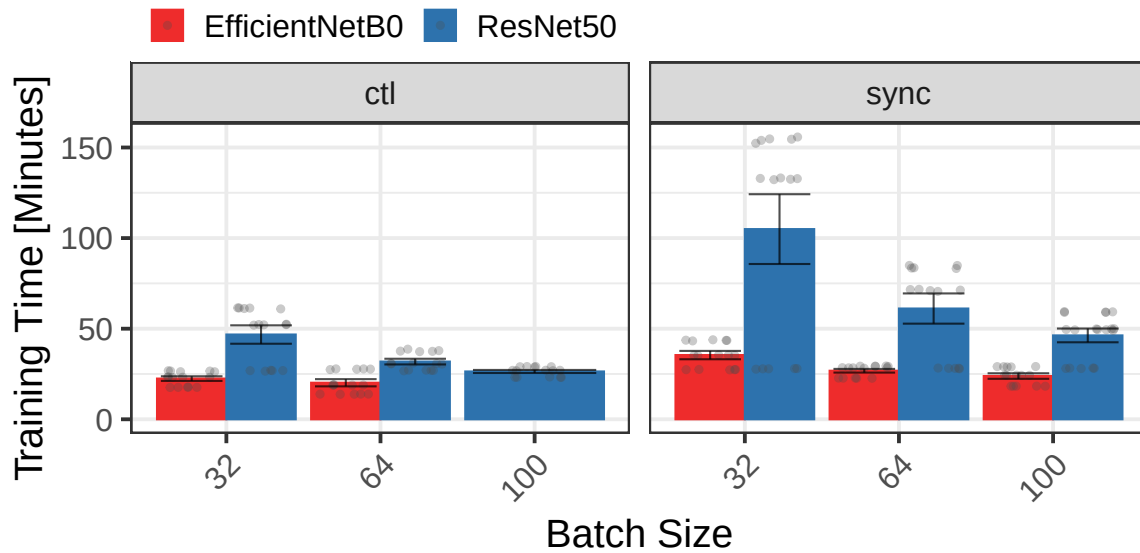


Figure 3. Mean of experimental results grouped by batch size.

like to thank the PCAD at INF/UFRGS for making infrastructure and hardware used in the experiments available.

References

- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Deep residual learning for image recognition. *CoRR*, abs/1512.03385.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946.